

FC7xxx FLS Integration Manual

Rev.0.1

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0

Table of Contents

Revision History	2
Table of Contents	3
Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive Area	9
Chapter 5 Error Report	10
5.1 Det	10
5.2 Dem	10
Chapter 6 Function Calls	11
6.1 Function Calls during Startup	11
6.2 Function Calls during Shutdown	11
6.3 Function Calls during Wake-up	11
6.4 Function Calls during Runtime	11
Chapter 7 Other Requirements	12
7.1 Notification, Callback, Callout	12
Chapter 8 Integration Steps	13

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the FLS module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Common: This module is the basic module which used to choose the chip.

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for enabling Development error detection.

2.2 Files Required for Compile

FLS module files:

- _MCAL/Src/Fls/Src/Fls.c
- _MCAL/Src/Fls/Src/Fls_Flash.c
- _MCAL/Src/Fls/Src/ Fls_IPW.c
- _MCAL/Src/Fls/include/Fls.h
- _MCAL/Src/Fls/include/Fls_Flash.h
- _MCAL/Src/Fls/include/Fls_IPW.h
- _MCAL/Src/Fls/include/Fcfmc_Regs.h
- _MCAL/Src/Fls/include/Fls_Driver.h
- _MCAL/Src/Fls/include/Fls_Flash_Types.h
- _MCAL/Src/Fls/include/Fls_MemMap.h
- _MCAL/Src/Fls/include/Fls_Types.h
- _MCAL/Src/Fls/include/Fls_version.h

- _MCAL_generate/src/Fls_PBcfg.c
- _MCAL_generate/include/Fls_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h

Det module files:

- Det.h

Rte module files:

- SchM_Fls.h

2.3 Add Plug-ins

FLS module plug-ins are developed for EB tools, so, to use FLS plug-ins on the EB tool, the user needs to add the FLS module to the EB plug-ins folder first.

- 1) Copy FLS module (_MCAL/EB_Plugins/eclipse/plugins/ FLS) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
- 2) Set FLS module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure FLS module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
FLS_START_SEC_CONFIG_DATA_8 FLS_STOP_SEC_CONFIG_DATA_8 FLS_START_SEC_CONFIG_DATA_16 FLS_STOP_SEC_CONFIG_DATA_16 FLS_START_SEC_CONFIG_DATA_32 FLS_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
FLS_START_SEC_CONFIG_DATA_UNSPECIFIED FLS_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
FLS_START_SEC_CONST_BOOLEAN FLS_STOP_SEC_CONST_BOOLEAN FLS_START_SEC_CONST_8 FLS_STOP_SEC_CONST_8 FLS_START_SEC_CONST_16 FLS_STOP_SEC_CONST_16 FLS_START_SEC_CONST_32 FLS_STOP_SEC_CONST_32 FLS_START_SEC_CONST_UNSPECIFIED FLS_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
FLS_START_SEC_CODE FLS_STOP_SEC_CODE FLS_START_SEC_RAMCODE FLS_STOP_SEC_RAMCODE FLS_START_SEC_CODE_AC FLS_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
FLS_START_SEC_VAR_NO_INIT_BOOLEAN FLS_STOP_SEC_VAR_NO_INIT_BOOLEAN FLS_START_SEC_VAR_NO_INIT_8 FLS_STOP_SEC_VAR_NO_INIT_8 FLS_START_SEC_VAR_NO_INIT_16 FLS_STOP_SEC_VAR_NO_INIT_16 FLS_START_SEC_VAR_NO_INIT_32 FLS_STOP_SEC_VAR_NO_INIT_32 FLS_START_SEC_VAR_NO_INIT_UNSPECIFIED FLS_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
FLS_START_SEC_VAR_INIT_BOOLEAN FLS_STOP_SEC_VAR_INIT_BOOLEAN FLS_START_SEC_VAR_INIT_8 FLS_STOP_SEC_VAR_INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by

Section Name	Section Type	Description
FLS_START_SEC_VAR_INIT_16 FLS_STOP_SEC_VAR_INIT_16 FLS_START_SEC_VAR_INIT_32 FLS_STOP_SEC_VAR_INIT_32 FLS_START_SEC_VAR_INIT_UNSPECIFIED FLS_STOP_SEC_VAR_INIT_UNSPECIFIED		startup code (data).

Chapter 4 Exclusive Area

FLS module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the FLS driver:

- Fls_Flash.c:
 - Fls_Flash_Erase: exclusive area 0
 - Fls_Flash_Write: exclusive area 1
- Fls.c:
 - Fls_Erase: exclusive area 10
 - Fls_Write: exclusive area 11
 - Fls_Read : exclusive area 12
 - Fls_Compare: exclusive area 13
 - Fls_BlankCheck: exclusive area 14

Chapter 5 Error Report

5.1 Det

Function Name	Error Type
Fls_Init	FLS_E_PARAM_CONFIG FLS_E_BUSY
Fls_Erase	FLS_E_UNINIT FLS_E_PARAM_ADDRESS FLS_E_PARAM_LENGTH FLS_E_BUSY
Fls_Write	FLS_E_UNINIT FLS_E_PARAM_ADDRESS FLS_E_PARAM_LENGTH FLS_E_PARAM_DATA FLS_E_BUSY
Fls_Cancel	FLS_E_UNINIT
Fls_GetJobResult	FLS_E_UNINIT
Fls_Read	FLS_E_UNINIT FLS_E_PARAM_ADDRESS FLS_E_PARAM_LENGTH FLS_E_PARAM_DATA FLS_E_BUSY
Fls_Compare	FLS_E_UNINIT FLS_E_PARAM_ADDRESS FLS_E_PARAM_LENGTH FLS_E_PARAM_DATA FLS_E_BUSY
Fls_BlankCheck	FLS_E_UNINIT FLS_E_PARAM_ADDRESS FLS_E_PARAM_LENGTH FLS_E_BUSY
Fls_SetMode	FLS_E_UNINIT FLS_E_BUSY
Fls_GetVersionInfo	FLS_E_PARAM_POINTER
Fls_MainFunction	FLS_E_UNINIT
Fls_Flash_VerifyErase	FLS_E_VERIFY_ERASE_FAILED
Fls_Flash_VerifyWrite	FLS_E_VERIFY_WRITE_FAILED

5.2 Dem

None

Chapter 6 Function Calls

6.1 Function Calls during Startup

The API needs to be called is `Fls_Init(ConfigPtr);`

6.2 Function Calls during Shutdown

None

6.3 Function Calls during Wake-up

None

6.4 Function Calls during Runtime

The API needs to be called is `Fls_MainFunction ();`

Chapter 7 Other Requirements

7.1 Notification, Callback, Callout

- Notification

If the user configures notifications and the value is not "NULL_PTR" or "NULL", an extern declaration will generate in Fls_Cfg.h. User need implement the notification in any file which include Fls_Cfg.h.

- Callback

None

- Callout

None

Chapter 8 Integration Steps

- 1) Configure the FLS module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the FLS module with dependent modules.